

**BABU BANARASI DAS
UNIVERSITY
2026-2027**

SCHOOL OF COMPUTER APPLICATIONS

**NoSQL Project On
Unique Clothing Management System Using MongoDB**

Submitted to:

Mr. Harendra Singh

Submitted by:

student's name

Anushka Maurya

Department: BCA (DS & AI)

Roll no:31

Date: 25th September

university roll no: 1250258115

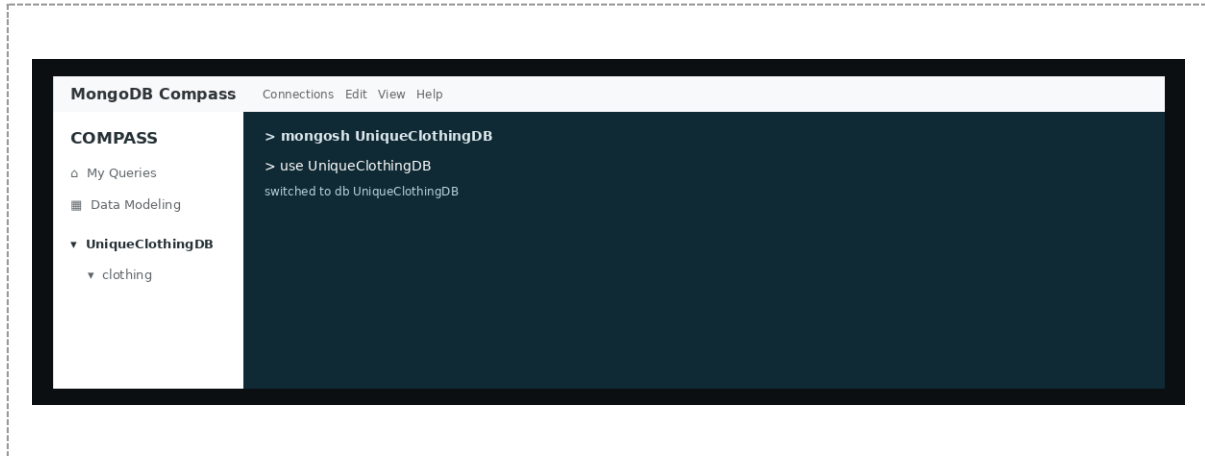
1. CREATE DATABASE

MongoDB Query:

use UniqueClothingDB

Explanation: Creates or switches to the UniqueClothingDB database.

Output Screenshot:



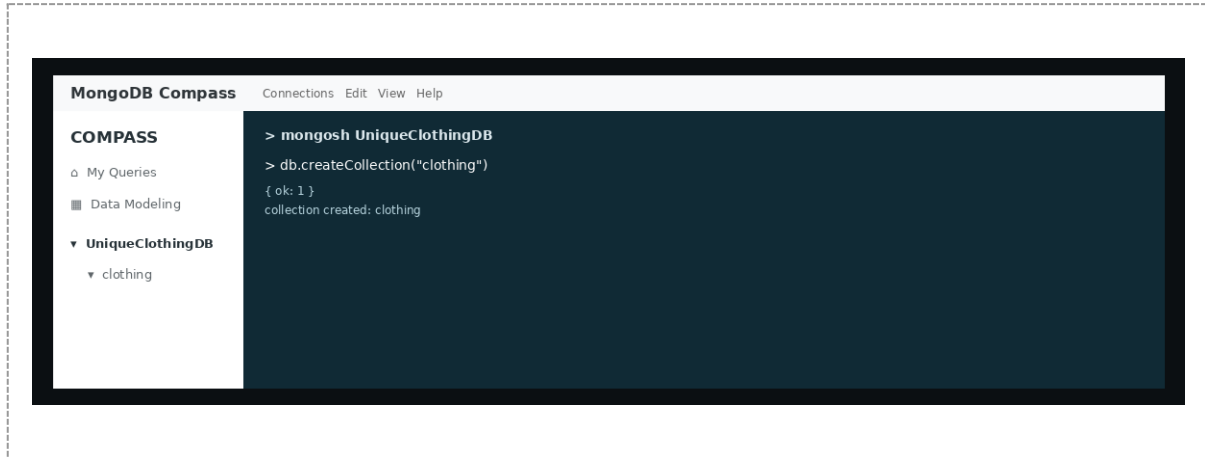
2. CREATE COLLECTION

MongoDB Query:

db.createCollection("clothing")

Explanation: Creates a collection named clothing.

Output Screenshot:



3. INSERT 50 CLOTHING PRODUCTS

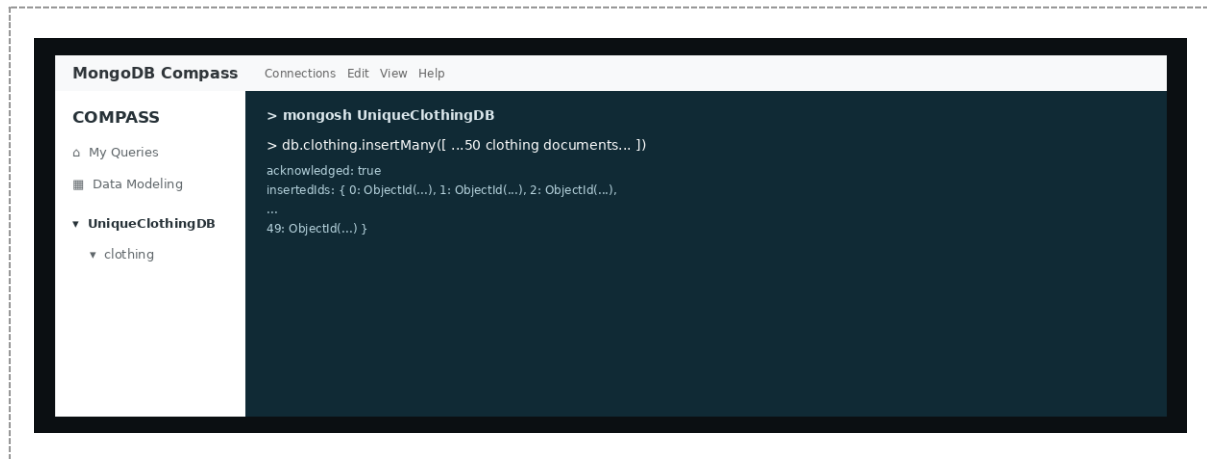
MongoDB Query:

```
db.clothing.insertMany([
  {clothingId:1, name:"Aarav", price:62000, category:"Jackets", size:"S"},
  {clothingId:2, name:"Bhumi", price:60000, category:"Hoodies", size:"M"},
  {clothingId:3, name:"Chetan", price:47000, category:"Shirts", size:"L"},
  {clothingId:4, name:"Damini", price:44500, category:"Jeans", size:"XL"},
  {clothingId:5, name:"Eshan", price:78000, category:"T-Shirts", size:"XXL"},
  {clothingId:6, name:"Falguni", price:83000, category:"Jackets", size:"S"},
  {clothingId:7, name:"Gopal", price:31000, category:"Dresses", size:"M"},
  {clothingId:8, name:"Harleen", price:80000, category:"Hoodies", size:"L"},
  {clothingId:9, name:"Ishita", price:76000, category:"Shirts", size:"XL"},
  {clothingId:10, name:"Jayesh", price:48000, category:"Jeans", size:"XXL"},
  {clothingId:11, name:"Kajal", price:63000, category:"Jackets", size:"S"},
  {clothingId:12, name:"Lavanya", price:49500, category:"Hoodies", size:"M"},
  {clothingId:13, name:"Mohit", price:92000, category:"Shirts", size:"L"},
  {clothingId:14, name:"Naina", price:36000, category:"Jeans", size:"XL"},
  {clothingId:15, name:"Omkar", price:80000, category:"T-Shirts", size:"XXL"},
  {clothingId:16, name:"Palak", price:66000, category:"Jackets", size:"S"},
  {clothingId:17, name:"Rishabh", price:57500, category:"Dresses", size:"M"},
  {clothingId:18, name:"Sakshi", price:93000, category:"Hoodies", size:"L"},
  {clothingId:19, name:"Tushar", price:79500, category:"Shirts", size:"XL"},
  {clothingId:20, name:"Urvashi", price:91000, category:"Jeans", size:"XXL"},
  {clothingId:21, name:"Vansh", price:56000, category:"Jackets", size:"S"},
  {clothingId:22, name:"Yamini", price:53000, category:"Hoodies", size:"M"},
  {clothingId:23, name:"Zoya", price:94000, category:"Shirts", size:"L"},
  {clothingId:24, name:"Abhay", price:63500, category:"Jeans", size:"XL"},
  {clothingId:25, name:"Bhaskar", price:88000, category:"T-Shirts", size:"XXL"},
  {clothingId:26, name:"Charu", price:70500, category:"Jackets", size:"S"},
  {clothingId:27, name:"Dhruv", price:40000, category:"Dresses", size:"M"},
  {clothingId:28, name:"Eknoor", price:69500, category:"Hoodies", size:"L"},
  {clothingId:29, name:"Farida", price:90000, category:"Shirts", size:"XL"},
  {clothingId:30, name:"Girish", price:66000, category:"Jeans", size:"XXL"},
  {clothingId:31, name:"Heena", price:44000, category:"Jackets", size:"S"},
  {clothingId:32, name:"Imran", price:84000, category:"Hoodies", size:"M"},
  {clothingId:33, name:"Jyoti", price:86000, category:"Shirts", size:"L"},
  {clothingId:34, name:"Kunal", price:47000, category:"Jeans", size:"XL"},
  {clothingId:35, name:"Lakshmi", price:47000, category:"T-Shirts", size:"XXL"},
  {clothingId:36, name:"Madhav", price:29000, category:"Jackets", size:"S"},
  {clothingId:37, name:"Nisha", price:37000, category:"Dresses", size:"M"},
  {clothingId:38, name:"Onkar", price:80000, category:"Hoodies", size:"L"},
  {clothingId:39, name:"Prachi", price:63500, category:"Shirts", size:"XL"},
  {clothingId:40, name:"Radhika", price:55500, category:"Jeans", size:"XXL"},
  {clothingId:41, name:"Saurabh", price:60000, category:"Jackets", size:"S"},
  {clothingId:42, name:"Trisha", price:69500, category:"Hoodies", size:"M"},
  {clothingId:43, name:"Utkarsh", price:39500, category:"Shirts", size:"L"},
  {clothingId:44, name:"Vidhi", price:52000, category:"Jeans", size:"XL"},
  {clothingId:45, name:"Wasim", price:25000, category:"T-Shirts", size:"XXL"},
  {clothingId:46, name:"Yuvraj", price:44000, category:"Jackets", size:"S"},
  {clothingId:47, name:"Zeenat", price:39500, category:"Dresses", size:"M"},
  {clothingId:48, name:"Ankit", price:84000, category:"Hoodies", size:"L"}
])
```

```
{clothingId:49, name:"Bhawna", price:62500, category:"Shirts", size:"XL"}  
{clothingId:50, name:"Chandan", price:43500, category:"Jeans", size:"XXL"}  
])
```

Explanation: Inserts multiple clothing product documents into the collection.

Output Screenshot:



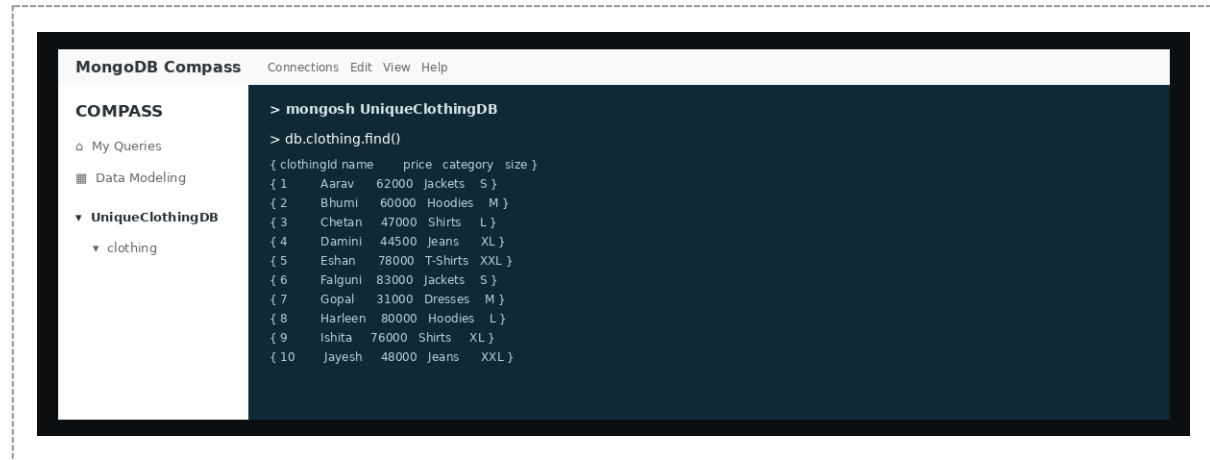
4. DISPLAY ALL CLOTHING PRODUCTS

MongoDB Query:

db.clothing.find()

Explanation: Retrieves all documents from the clothing collection.

Output Screenshot:



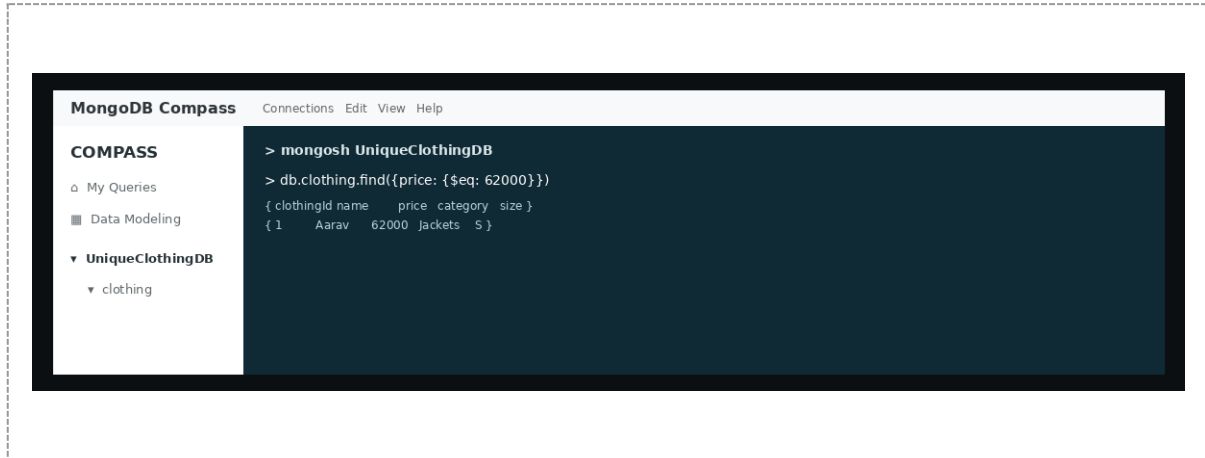
5. EQUAL TO — \$eq

MongoDB Query:

```
db.clothing.find({price: {$eq: 62000}})
```

Explanation: Finds clothing products whose price is exactly 62000.

Output Screenshot:



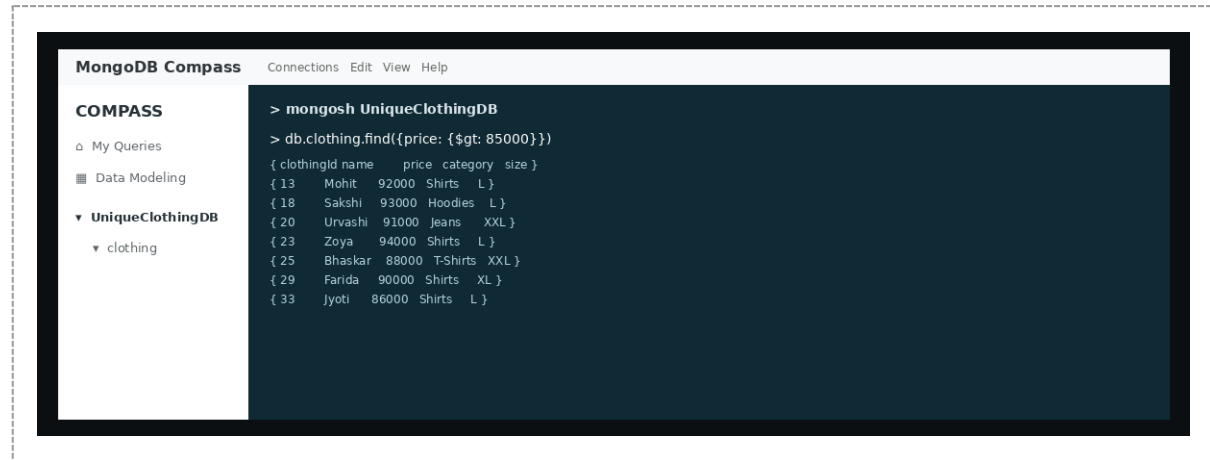
6. GREATER THAN — \$gt

MongoDB Query:

```
db.clothing.find({price: {$gt: 85000}})
```

Explanation: Finds clothing products whose price is greater than 85000.

Output Screenshot:



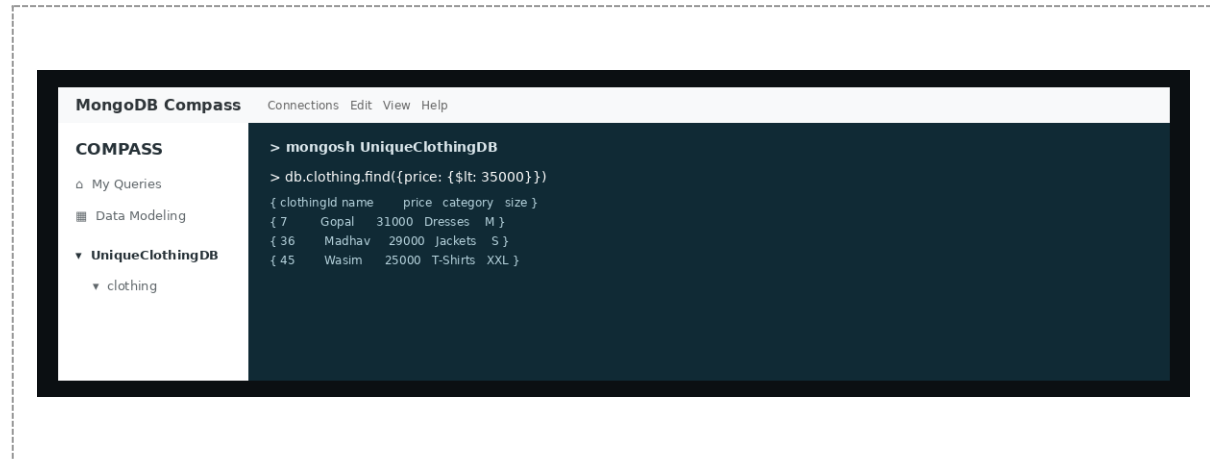
7. LESS THAN — \$lt

MongoDB Query:

db.clothing.find({price: {\$lt: 35000}})

Explanation: Finds clothing products whose price is less than 35000.

Output Screenshot:



8. GREATER THAN OR EQUAL TO — \$gte

MongoDB Query:

```
db.clothing.find({price: {$gte: 90000}})
```

Explanation: Finds clothing products whose price is 90000 or above.

Output Screenshot:



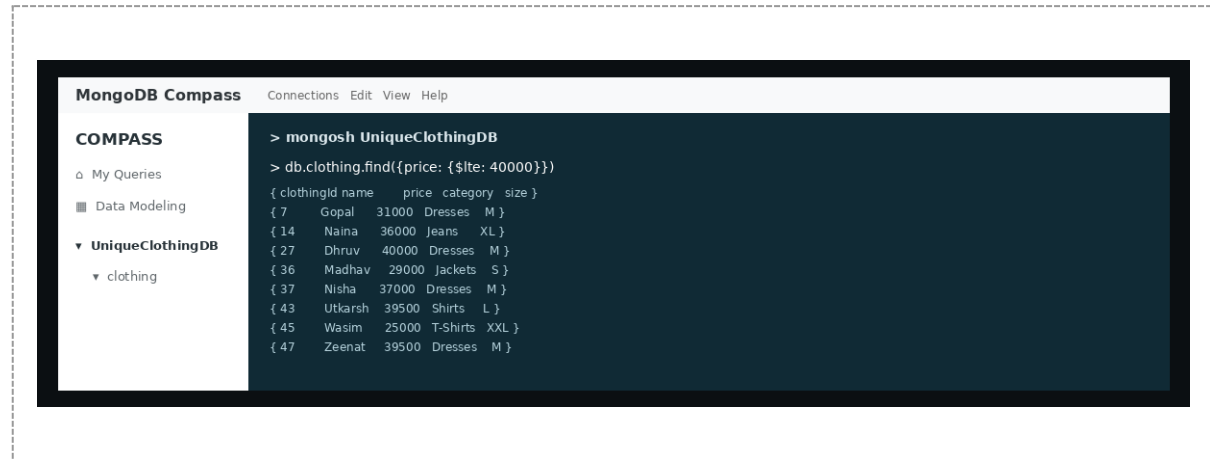
9. LESS THAN OR EQUAL TO — \$lte

MongoDB Query:

```
db.clothing.find({price: {$lte: 40000}})
```

Explanation: Finds clothing products whose price is 40000 or below.

Output Screenshot:



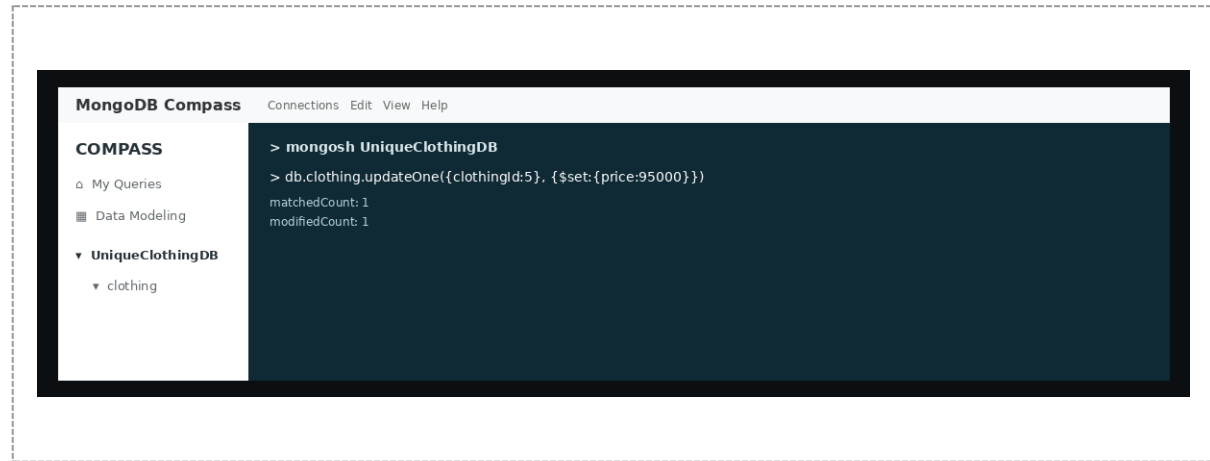
10. UPDATE OPERATION

MongoDB Query:

```
db.clothing.updateOne(  
  {clothingId:5},  
  {$set:{price:95000}}  
)
```

Explanation: Updates the price to 95000 of the clothing product with clothingId 5.

Output Screenshot:



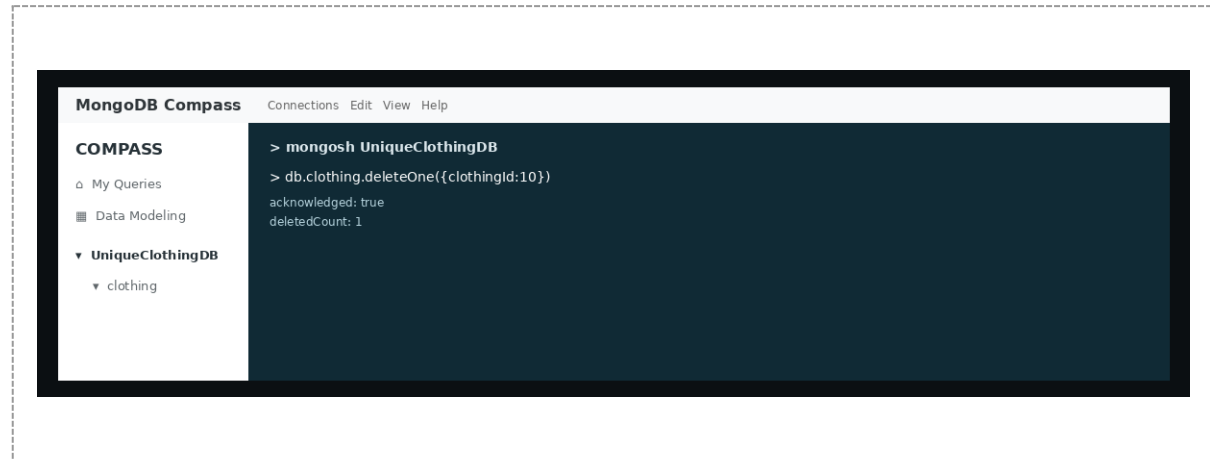
11. DELETE ONE OPERATION

MongoDB Query:

db.clothing.deleteOne({clothingId:10})

Explanation: Deletes one document matching clothingId 10.

Output Screenshot:



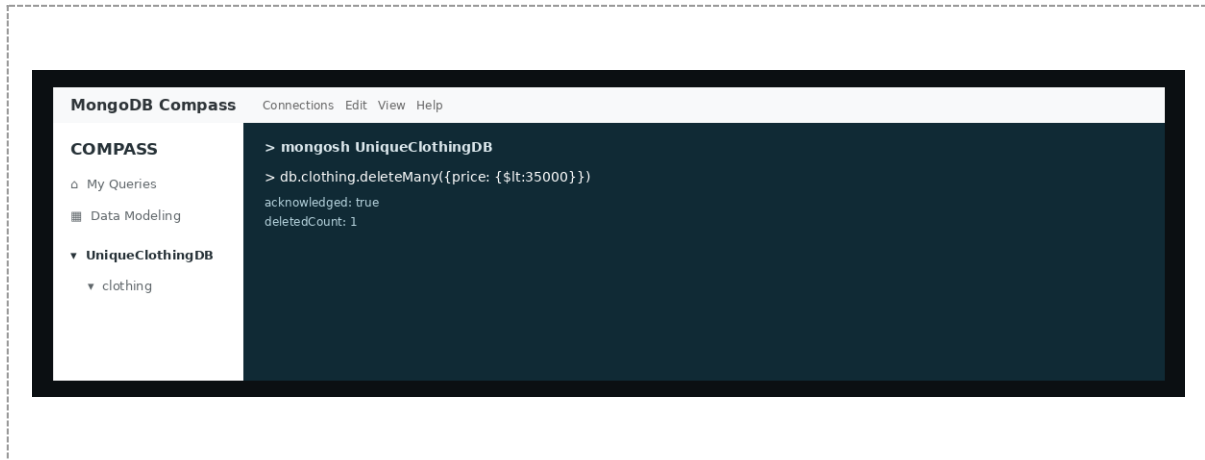
12. DELETE MANY OPERATION

MongoDB Query:

```
db.clothing.deleteMany({  
  price: {$lt:35000}  
})
```

Explanation: Deletes all clothing products having a price below 35000.

Output Screenshot:



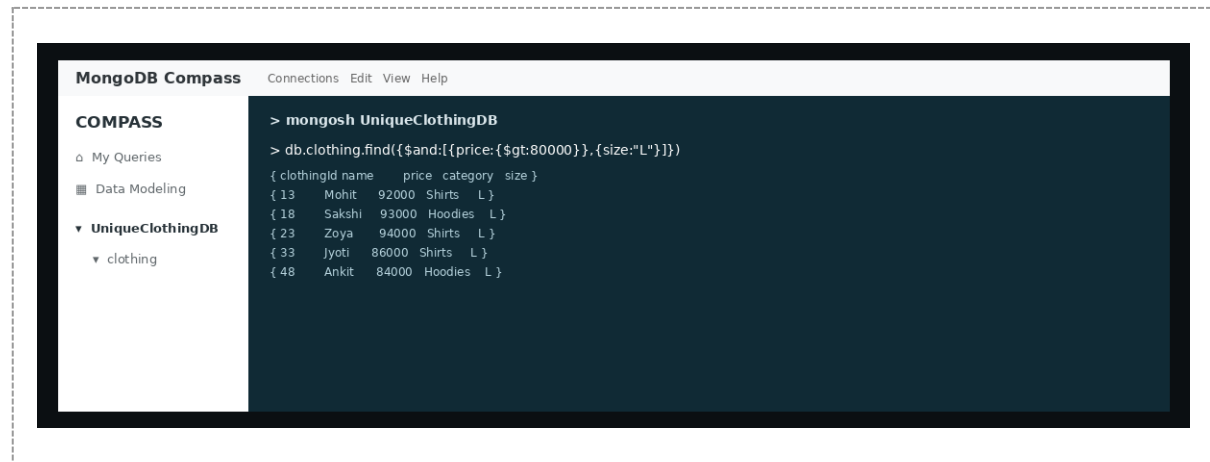
13. AND OPERATION — \$and

MongoDB Query:

```
db.clothing.find({
  $and:[
    {price:{$gt:80000}},
    {category:"Jackets"}
  ]
})
```

Explanation: Finds clothing products satisfying both conditions.

Output Screenshot:



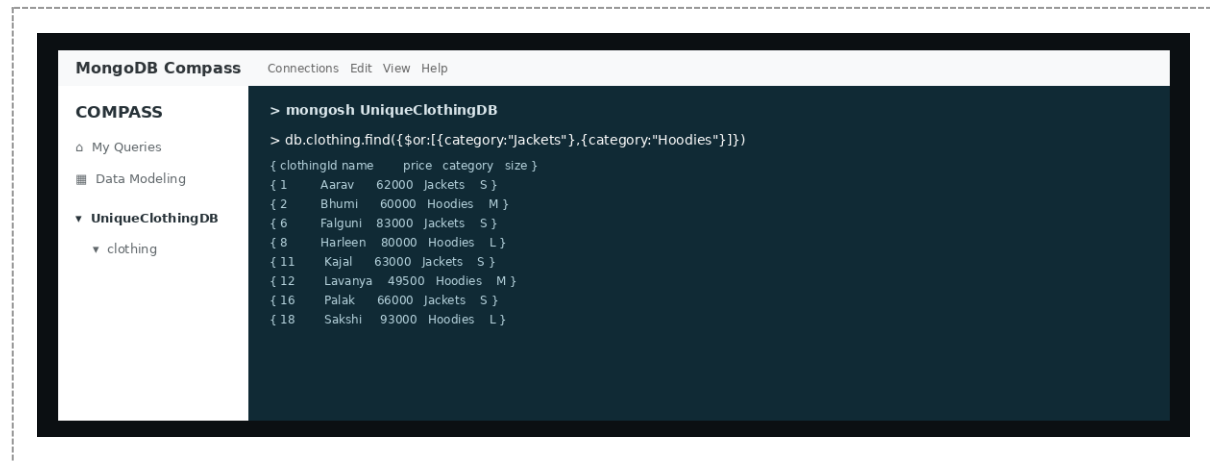
14. OR OPERATION — \$or

MongoDB Query:

```
db.clothing.find({
  $or:[
    {category:"Jackets"},
    {category:"Hoodies"}
  ]
})
```

Explanation: Finds clothing products in the Jackets or Hoodies category.

Output Screenshot:



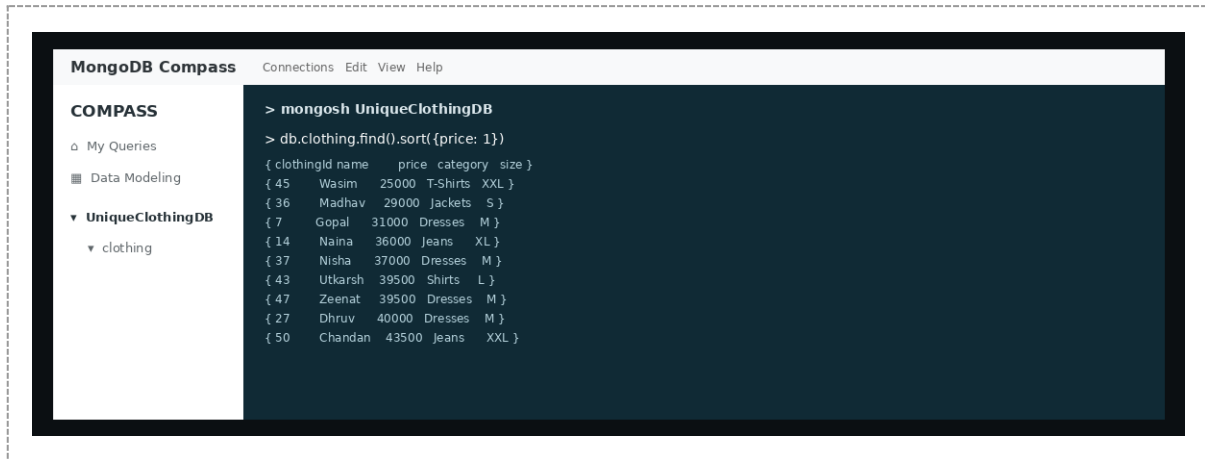
15. SORT OPERATION

A. MongoDB Query:

db.clothing.find().sort({price: 1})

Explanation: Sorts clothing products by price from lowest to highest. Use 1 for ascending order.

Output Screenshot:

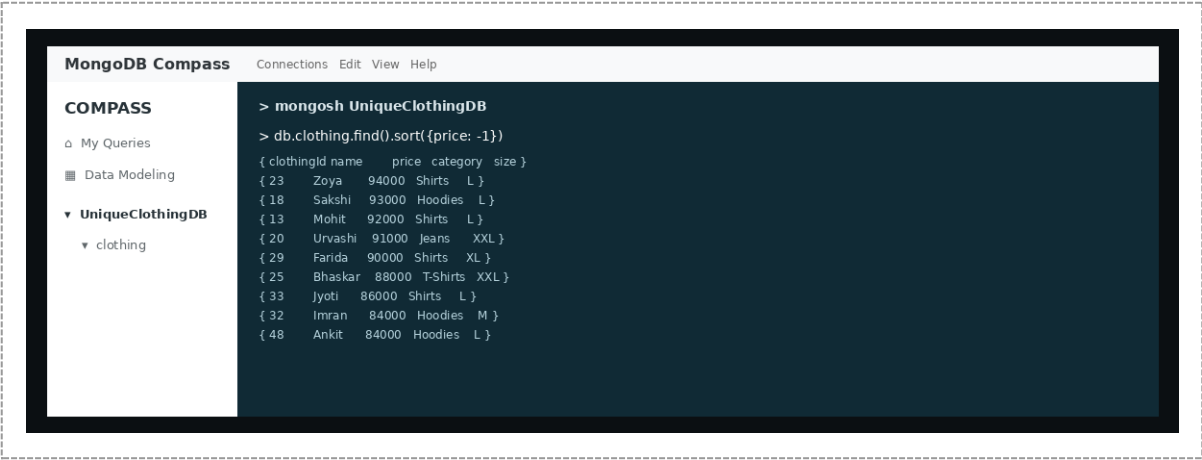


B. MongoDB Query:

db.clothing.find().sort({price: -1})

Explanation: Sorts clothing products by price from highest to lowest. Use -1 for descending order.

Output Screenshot:



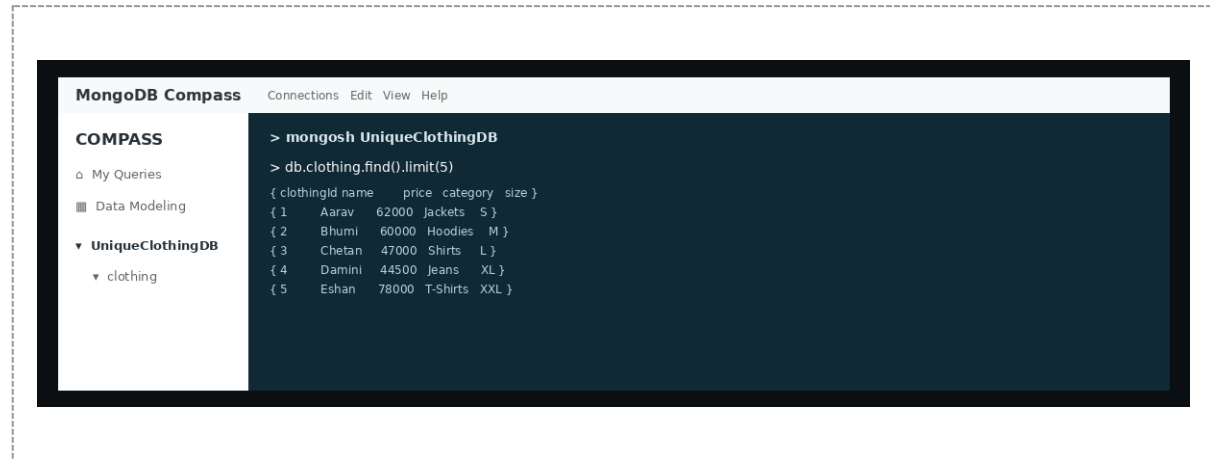
16. LIMIT OPERATION

MongoDB Query:

db.clothing.find().limit(5)

Explanation: Displays only the first five clothing products.

Output Screenshot:



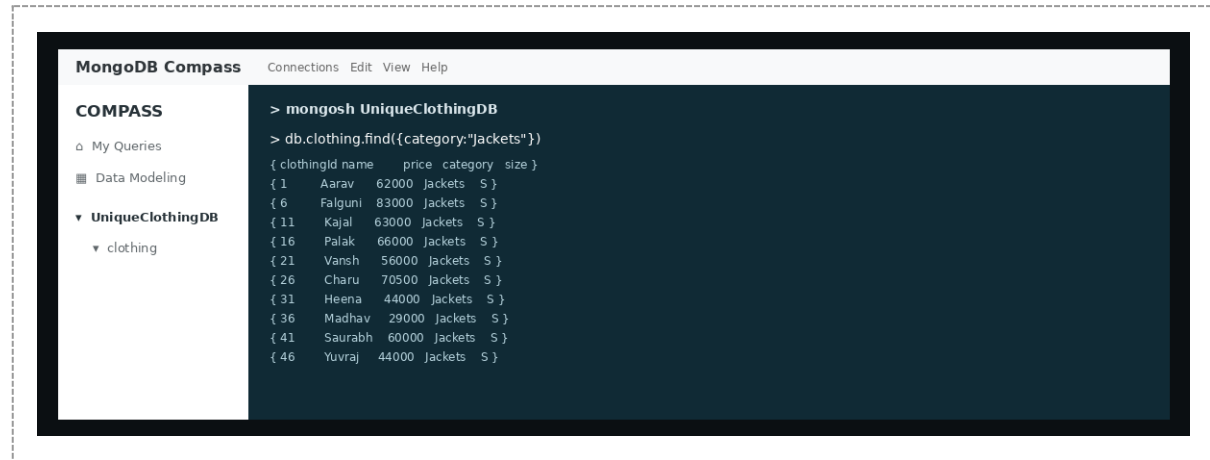
17. SEARCH BY CATEGORY

MongoDB Query:

db.clothing.find({category:"Jackets"})

Explanation: Finds all clothing products in the Jackets category.

Output Screenshot:



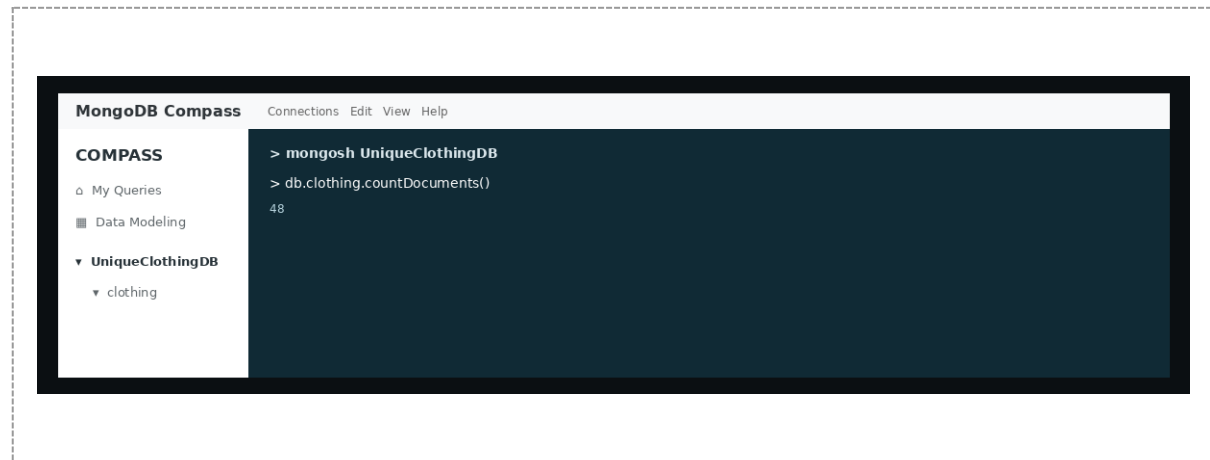
18. COUNT DOCUMENTS

MongoDB Query:

db.clothing.countDocuments()

Explanation: Counts clothing product documents currently present in the collection (performed after the delete operations).

Output Screenshot:



MONGODB OPERATORS USED

Operator	Meaning	Example
\$eq	Equal to	price: {\$eq:62000}
\$gt	Greater than	price: {\$gt:85000}
\$lt	Less than	price: {\$lt:35000}
\$gte	Greater than or equal to	price: {\$gte:90000}
\$lte	Less than or equal to	price: {\$lte:40000}
\$and	All conditions true	\$and:[...]
\$or	At least one condition true	\$or:[...]

CONCLUSION

This project provided practical knowledge of NoSQL databases using MongoDB. A Unique Clothing Management System containing clothing product information was created and different MongoDB operations were performed on the data.

Through this project, I learned how to insert, retrieve, update, and delete documents. I also learned how to arrange data in ascending and descending order, how comparison operators such as \$eq, \$gt, \$lt, \$gte, and \$lte work, along with logical operators such as \$and and \$or.

Overall, this project helped me understand how MongoDB can be used to manage and query clothing product data efficiently.